FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

PROJECT REPORT

AI PLACEMENT MENTOR

A PROJECT REPORT

Submitted by

DURAIMADHAN MM – 240701131

GOKUL S – 240701148



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report “AI PLACEMENT MENTOR” is the bonafide work of “DURAIMADHAN MM (240701131) and GOKUL S (240701148)” who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Mrs. B. Deepa

Assistant Professor (SG)

Department of Computer Science and Engineering

Rajalakshmi Engineering College

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	1
1	INTRODUCTION	2
2	LITERATURE REVIEW	4
3	PROPOSED SYSTEM	7
4	MODULES DESCRIPTION	15
5	IMPLEMENTATION AND RESULTS	19
6	CONCLUSION AND FUTURE WORK	25
7	REFERENCES	26

TABLE OF FIGURES

FIGURE NO	TITLE	PAGE NO
Fig. 3.1	System architecture of AI Placement Mentor	7
Fig. 3.2	Logical data model	9
Fig. 3.3	Flowchart of the resume analysis pipeline	10
Fig. 3.4	Constraint propagation and ranking for company matching (illustrative example)	13
Fig. 4.1	Module overview of the system	15
Fig. 4.2	Interview answer evaluation with local fallback	17
Fig. 4.3	Tavus AI interview workflow	18
Fig. 5.1	Student dashboard (Home)	21
Fig. 5.2	Aptitude assessment interface	22
Fig. 5.3	Coding challenge and editor interface	22
Fig. 5.4	Mock interview answer with AI feedback	23
Fig. 5.5	Tavus AI interview session	23

TABLE OF TABLES

TABLE NUMBER	TITLE	PAGE NO
Table 2.1	Comparison of approaches	6
Table 3.1	Technology stack	8
Table 3.2	Resume analysis output fields	10
Table 3.3	Constraints used for company matching	11
Table 4.1	Authentication and status interfaces	16
Table 5.1	Software environment	19
Table 5.2	Dataset used for the experiment	20
Table 5.3	Configuration and run details	20

ABSTRACT

Preparing for campus recruitment is difficult for many students. They rarely know how well their resume matches real job requirements, which roles suit their skills, or how they will perform in an interview, and personal feedback from mentors is limited by time. This project presents AI Placement Mentor, a full-stack placement preparation platform that guides a student from resume upload to mock interview practice. A FastAPI back end parses an uploaded PDF resume, extracts contact details and skills, and reports strengths, weaknesses, missing competencies and recommended roles. The extracted skills are then matched against a database of companies and job roles using skill similarity and a constraint-propagation algorithm, which removes company roles whose core requirements are not met and ranks the remaining ones. A mock interview engine supports aptitude, coding, technical, behavioural and communication rounds. Answers are evaluated by the OpenAI Chat Completions API when a key is configured and by a local keyword-and-quality evaluator otherwise, so the platform keeps working without external access. A Tavus integration starts a conversational AI interviewer with a custom persona. The system is implemented with Next.js, React and TypeScript on the front end and FastAPI, SQLAlchemy and SQLite on the back end.

Keywords: resume analysis, constraint propagation, company matching, mock interview, FastAPI, Next.js, OpenAI, Tavus.

CHAPTER 1

INTRODUCTION

1.1 Background

Campus recruitment is a decisive stage in a student's career. Before an offer is made, a student usually passes through several steps: resume screening, an aptitude test, a coding round, a technical interview and a behavioural or communication round. Each step tests a different ability, and each company asks for a different combination of skills.

Most students prepare for these steps with little feedback. A resume is often written once and never checked against real job requirements, so a student may apply to roles for which a mandatory skill is missing, or overlook roles that would suit the profile well. Mock interviews depend on the time of seniors and faculty, which is limited near the placement season, and generic online advice is not tied to the student's own resume.

Artificial intelligence can close part of this gap. Language models can read an answer and comment on it, and classical AI methods can reason over skill requirements in a transparent way. The Foundations of Artificial Intelligence course introduces constraint satisfaction, constraint propagation and knowledge-based reasoning. This project applies those ideas, together with a modern language model, to a practical placement assistant.

1.2 Problem Statement

Given a resume in PDF format, a set of companies in which each company role has a list of required skills, and a set of interview round types, the system must extract the skills and contact details from the resume, assess strengths and weaknesses, recommend suitable roles and produce a ranked list of company matches. It must also let the student practise interview rounds and receive a score and written feedback on each answer. The system must remain usable when the external language-model service is not available, and it must let each student create an account and sign in.

1.3 Objectives

- To build a web platform in which a student can register, sign in and upload a resume in PDF format.

- To extract skills and contact details from the resume and report strengths, weaknesses and recommended roles.
- To match the resume against stored companies and roles using skill similarity and a constraint-propagation algorithm, and to rank the results.
- To support mock interview rounds in aptitude, coding, technical, behavioural and communication areas, with a score and feedback for each answer.
- To evaluate answers with the OpenAI API when it is available and with a local evaluator when it is not, so that the platform works in both cases.
- To start a conversational AI interview with a configurable interviewer persona through the Tavus API.
- To present all of these features in a responsive dashboard for the student.

1.4 Scope and Contributions

The system covers one deployment of the platform for students who prepare for campus recruitment. Resumes are accepted in PDF format, company data is stored in a seeded SQLite database, and the interface is designed for the student role. The main contributions of this work are:

- An end-to-end pipeline that takes a PDF resume and returns skills, strengths, weaknesses, recommended roles and ranked company matches.
- A company-matching step that applies constraint propagation to remove unsuitable company roles before the rest are ranked by skill similarity.
- A dual-mode answer evaluator that uses the OpenAI Chat Completions API when available and falls back to a local keyword-and-quality evaluator.
- An integration with the Tavus API that starts a conversational interview with a custom AI interviewer persona.
- A responsive dashboard with Home, Resume Analysis and Mock Interviews sections.

The rest of this report is organised as follows. Chapter 2 reviews related work, Chapter 3 describes the proposed system, Chapter 4 explains each module, Chapter 5 presents the setup and results, and Chapter 6 concludes the report and lists future work.

CHAPTER 2

LITERATURE REVIEW

This chapter surveys work on resume analysis and job matching, constraint satisfaction, language-model based evaluation and robust service design, which are the four areas this project draws on.

2.1 Resume Analysis and Job Matching

Matching a person's profile to a set of items is a standard problem in information retrieval. Jones (1972) introduced inverse document frequency, which gives more weight to rare terms, and Salton and Buckley (1988) compared term-weighting schemes for automatic text retrieval. These ideas underlie most keyword-based and similarity-based matching, including the matching of a resume to a job description.

Ricci, Rokach and Shapira (2015) describe content-based recommendation, in which the attributes of an item are compared with a profile of the user. Matching the skills in a resume with the skills required by a company role is a content-based recommendation of this kind. A weakness of similarity alone is that it can rank a role highly even when one mandatory requirement is missing, because a high overlap on other skills hides the gap. This project therefore adds a constraint step before ranking.

2.2 Constraint Satisfaction and Propagation

Russell and Norvig (2020) define a constraint satisfaction problem by a set of variables, a domain of values for each variable and a set of constraints. Besides search, they describe constraint propagation, in which the constraints are used to remove values from the domains that cannot be part of any solution, so that later steps work on a smaller problem.

The theoretical basis comes from earlier work. Mackworth (1977) introduced consistency algorithms for networks of relations, which led to the study of constraint propagation. Tsang (1993) and Dechter (2003) give book-length treatments of constraint satisfaction and constraint processing. In this project the same idea is used in a simple form: company roles whose core skill requirements are not satisfied by the resume are removed from the candidate set before any ranking is done.

2.3 Language Models for Evaluating Answers

Brown et al. (2020) showed that large language models can perform new tasks from a few examples in the prompt, which makes them usable for tasks such as judging a written answer without task-specific training. OpenAI (2023) reported the capabilities of GPT-4 on professional and academic benchmarks.

Zheng et al. (2023) studied the use of strong language models as judges of other model outputs. They reported that a strong model such as GPT-4 agrees with human judgements on more than 80% of comparisons, about as often as humans agree with each other, but they also described biases such as a preference for longer answers. These findings support using a language model to give feedback on interview answers. They also show why a simple and transparent local check is useful as a fallback and as a sanity filter.

2.4 Service Design and Graceful Degradation

The platform is built as a REST service, an architectural style described by Fielding (2000) in which resources are accessed through a uniform interface. Jones, Bradley and Sakimura (2015) define the JSON Web Token format, which is widely used to carry authentication claims between a client and a server. Bayer (2012) describes SQLAlchemy, the object-relational mapper used here to store data in a relational database.

A further design principle is graceful degradation: a feature that depends on an external service should still return a useful result when that service is unavailable. The mock interview evaluator follows this principle by falling back to local scoring when the OpenAI service cannot be used.

2.5 Summary and Research Gap

Table 2.1 Comparison of approaches

Approach	Strength	Limitation	Use in this project
Advice from seniors or the placement cell	Personal and experienced	Limited by time and not available on demand	Complemented by automated feedback
Keyword-only resume screening	Simple and fast	Ignores mandatory requirements and gives no reason	Replaced by propagation plus similarity
Constraint propagation with similarity ranking	Respects core requirements, ranks the rest, easy to explain	Needs well-structured skill lists for each role	Core of company matching
Language model used alone	Natural feedback on free text	Needs a key and network access; may be inconsistent	Used when available, with a local fallback

Many placement tools address a single step, such as resume checking or question practice. Few combine resume analysis, requirement-aware company matching, multi-round interview practice and a conversational interviewer in one workflow that keeps working without an external AI service. This project addresses that gap by integrating all of these in one platform.

CHAPTER 3

PROPOSED SYSTEM

The proposed system is a three-tier web application. A Next.js front end talks to a FastAPI back end over a REST interface that exchanges JSON. The back end calls four intelligence components, namely the resume agent, the company-matching algorithm, the interview evaluator and the Tavus client, and stores its data in a SQLite database through SQLAlchemy.

3.1 Architecture Diagram

Figure 3.1 shows the overall architecture. A student works through one Next.js application. The application sends requests to the FastAPI back end, which calls the intelligence components and the database and returns the result as JSON.

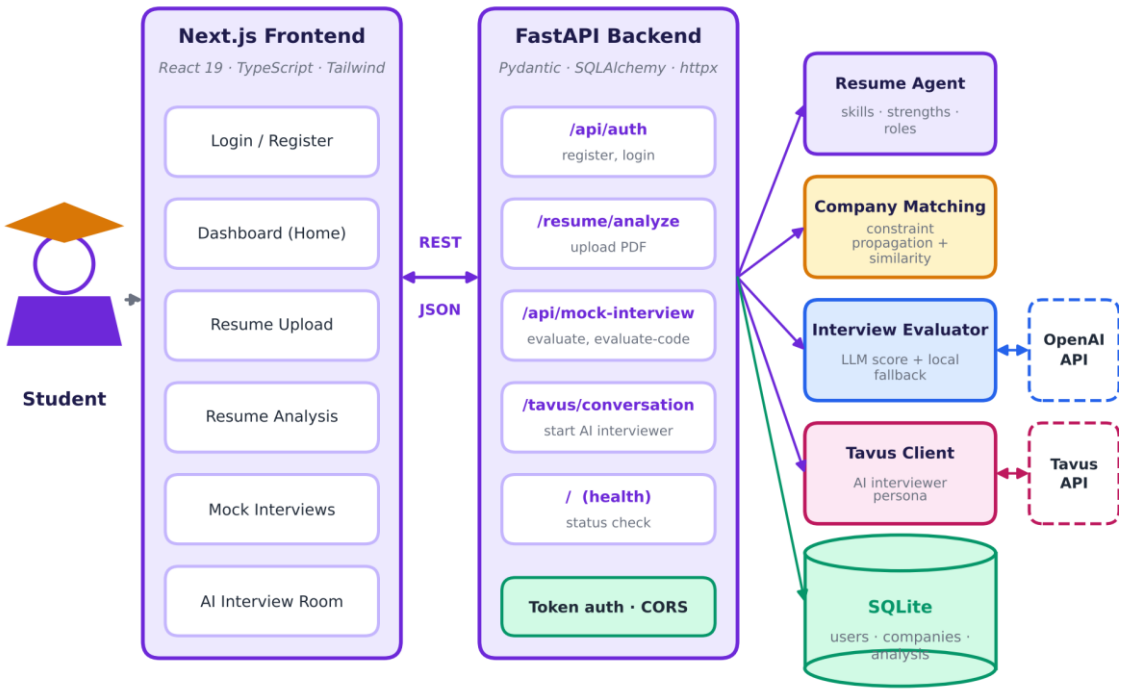


Fig. 3.1 System architecture of AI Placement Mentor

The architecture has four layers:

- **Presentation layer.** Next.js 16 with React 19, TypeScript and Tailwind CSS. Framer Motion adds animation and Lucide provides the icons. The pages cover the dashboard, resume upload and analysis, and the interview experiences.

- **Application layer.** FastAPI exposes route groups for authentication, resume analysis, mock interview evaluation and the Tavus integration, plus a root status route. Pydantic models validate every request.
- **Intelligence layer.** The resume agent analyses the resume, the constraint-propagation algorithm matches companies, the evaluator scores interview answers and the Tavus client starts conversational interviews.
- **Data layer.** SQLAlchemy with SQLite stores users, companies and resume analysis records in the file backend/data/placement.db.

Table 3.1 Technology stack

Layer	Technology	Purpose
Front end	Next.js 16, React 19, TypeScript, Tailwind CSS, Framer Motion, Lucide icons	User interface, pages and dashboard
Back end	Python, FastAPI, Pydantic, httpx, python-dotenv	REST API, validation, outgoing HTTP calls and configuration
Security	Token-based authentication	Sign-in and access to the API
Database	SQLite with SQLAlchemy	Persistent storage of users, companies and analyses
AI engine	Resume agent and constraint-propagation matcher	Skill analysis and company matching
Language model	OpenAI Chat Completions API, with local fallback	Evaluation of interview answers
Video interview	Tavus API	Conversational AI interviewer

3.2 Database Design

Figure 3.2 shows the logical tables and the main fields of each. The field names are given as key attributes and may differ slightly from the exact names in the code.

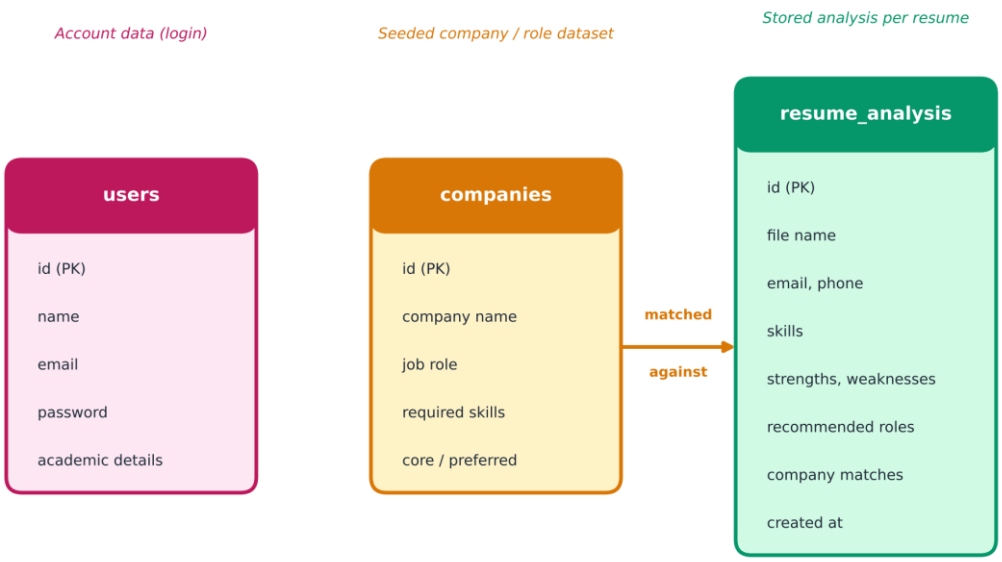


Fig. 3.2 Logical data model (key fields of each table)

The users table stores the account details entered at registration, including personal academic information. The companies table stores each company with its roles and the skills required for each role. It is filled by the seeding script seed_companies.py and is the only source of company data for matching. The resume_analysis table stores the result of each analysis, including the extracted contact details, skills, strengths, weaknesses, recommended roles and company matches.

The most important design decision is to keep company data in the database instead of in the code. The same records feed the matching algorithm and the recommendation text, so the company list can be extended by adding rows without changing the algorithm. The database path is currently set directly in database.py. Moving it to an environment variable is listed as future work.

3.3 Resume Analysis Design

When a student uploads a resume, the back end follows the steps in Figure 3.3. The upload is checked first, so that a file that is not a PDF or is empty is rejected before any analysis starts.

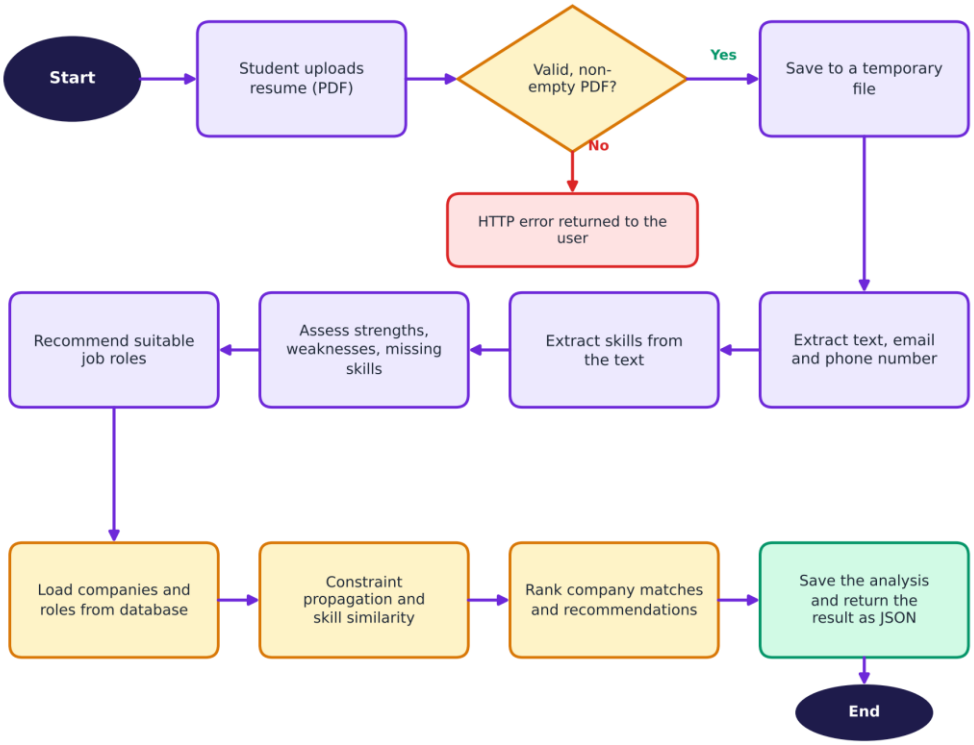


Fig. 3.3 Flowchart of the resume analysis pipeline

The analysis returns the fields listed in Table 3.2. The uploaded PDF is written to a temporary file for parsing, and the analysis result is then saved to the database.

Table 3.2 Resume analysis output fields

Field	Description
Email and phone	Contact details found in the resume text
Skills	Technical and professional skills extracted from the resume
Strengths	Areas in which the resume is strong, based on the extracted skills
Weaknesses	Areas that are weak or thin in the resume
Missing competencies	Skills usually expected for suitable roles that were not found
Recommended roles	Job roles that suit the skill profile
Company matches	Ranked company roles that fit the profile (Section 3.4)
Recommendations	Short advice derived from the match results

3.4 Company Matching Formulation

The matching step is formulated as a constraint problem, so that the reasons for keeping or removing a company role can be explained to the student.

Variables

Each company role stored in the database is a variable. Its value is either eligible or not eligible for the student who uploaded the resume.

Domains

Before propagation every variable can take both values. A company role becomes not eligible when a hard constraint fails. The skills in the resume and in the company data are compared after normalisation, so that differences in letter case or punctuation, for example between “Node.js” and “node js”, do not cause false mismatches.

Constraints

Table 3.3 Constraints used for company matching

ID	Constraint	Type
C1	Every core skill required by a role must appear among the normalised resume skills	Hard
C2	A resume with no extractable skills cannot be matched to any role	Hard
C3	Skill names are compared after normalising case and punctuation	Hard
C4	Preferred skills raise the score of a company role but never remove it	Soft
C5	Company roles with a higher skill similarity are ranked earlier	Soft

Hard constraints decide which company roles remain. Soft constraints only influence the order of the roles that remain.

3.5 Matching Algorithm

The matching step works in the following stages, which are also shown in Algorithm 3.1.

1. **Normalise.** Convert the resume skills and the company skills to a common form (C3).
2. **Load candidates.** Read all company roles from the database.
3. **Propagate constraints.** Remove every company role for which a core skill is missing from the resume (C1, C2).
4. **Score.** For each remaining role compute the similarity between the resume skills and the skills the role lists (C4, C5).
5. **Rank.** Sort the remaining roles by score, highest first.
6. **Recommend.** Return the ranked matches with short recommendations and store them with the analysis.

Algorithm 3.1 Constraint propagation and ranking for company matching

```
function MATCH_COMPANIES(resume_skills, roles):
   skills <- NORMALISE(resume_skills)
   if skills is empty:
      return empty list                 # C2
   candidates <- []
   for role in roles:
      if CORE_SKILLS(role) subset of skills:   # C1
         score <- SIMILARITY(skills, role)       # C4, C5
         add (role, score) to candidates
   sort candidates by score, highest first
   return candidates
```

Why propagation helps

A company role that fails a core requirement should never appear in the result, however well the other skills match. Removing such roles first keeps the final ranking simple, because every role that survives is one the student can reasonably apply for. It is the same idea as removing impossible values from a domain before searching in a constraint satisfaction problem. Figure 3.4 illustrates the two steps.

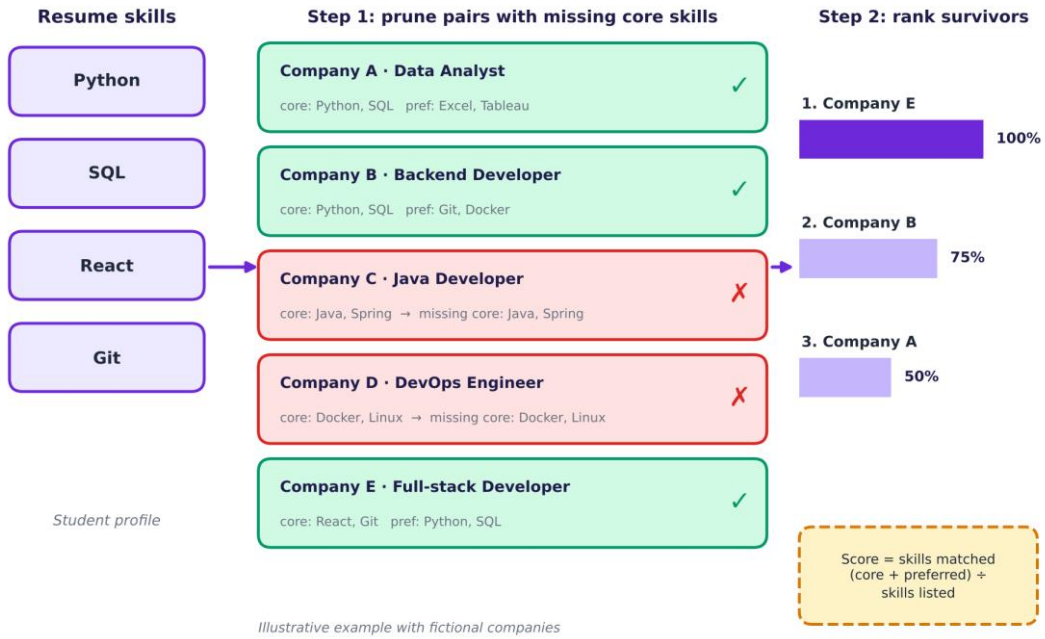


Fig. 3.4 Constraint propagation and ranking for company matching (illustrative example)

Effect of constraint propagation

Figure 3.4 uses five fictional company roles and a resume with the skills Python, SQL, React and Git. Company C and Company D are removed because their core skills (Java, Spring, Docker and Linux) are missing from the resume. The three remaining roles are ranked by the share of their listed skills that the resume covers. The scoring formula in the figure is a simplified illustration; the exact similarity measure is defined in the matching code.

Complexity

For R company roles with an average of k listed skills, normalisation, the core-skill test and the scoring each take time proportional to R × k, followed by a sort of the m surviving roles in O(m log m) time. The step is a single pass over the data and needs no backtracking, so its running time grows steadily with the size of the company dataset and stays small for a campus-sized list of companies.

3.6 Evaluation and Fallback Design

Interview answers are evaluated in two ways. A first filter rejects answers that are empty or very short, because they give too little evidence to score. For every other answer, the evaluator uses the OpenAI Chat Completions API when an API key is configured. If no

key is present, or the request fails, the local evaluator is used instead. It measures the overlap between the answer and the keywords expected for the question, applies quality heuristics, and returns a score with short feedback.

This design means that the platform never stops at the evaluation step because of a missing key or a network problem. It also keeps a simple and repeatable baseline that can be compared with the language-model score during testing. Because every evaluation returns written feedback together with the score, the student can see what to improve and not only how the answer was rated.

CHAPTER 4

MODULES DESCRIPTION

The system is divided into eight modules. Figure 4.1 shows how the student portal sits on top of a shared REST layer and the core services.

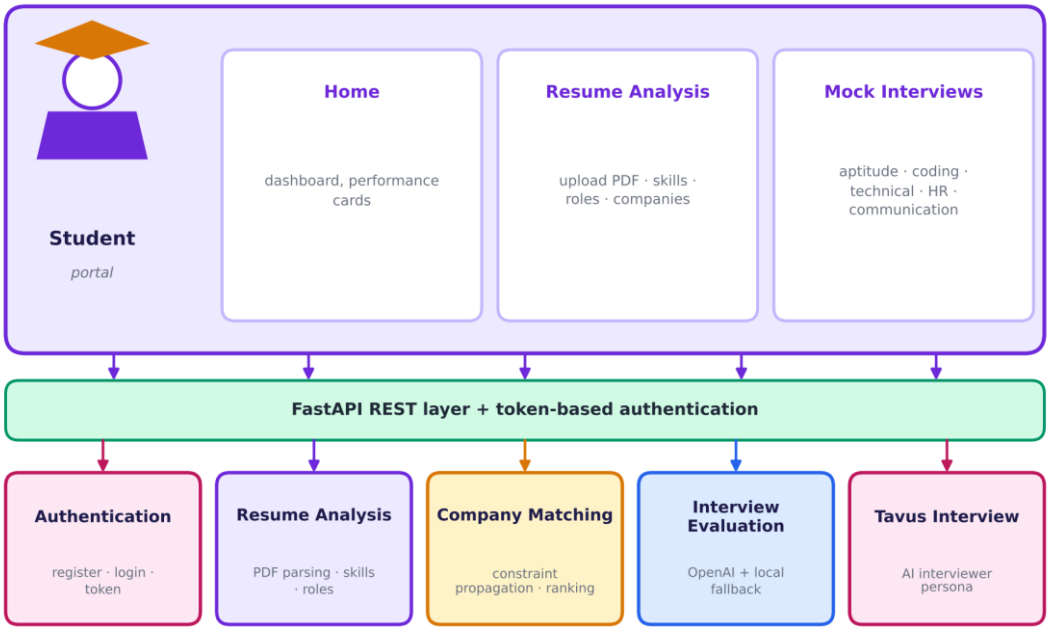


Fig. 4.1 Module overview of the system

4.1 Module 1: Authentication

This module lets a student create an account and sign in. Registration collects personal academic information together with the email and password. Login checks the email and password and returns an access token that the front end sends with later requests.

- **Input:** registration details (name, email, password and academic information), or the email and password for login.
- **Processing:** validation of the request with Pydantic models, lookup of the user in the database and creation of the access token.
- **Output:** a success message on registration, or an access token on login.

In the current prototype, passwords are stored without hashing. This is acceptable for a demonstration but not for production use, and password hashing is the first item in the future work of Chapter 6.

Table 4.1 Authentication and status interfaces

Method	Endpoint	Purpose
GET	/api/auth/test	Health check of the authentication API
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Log in and receive an access token
GET	/	Status check of the back end

4.2 Module 2: Resume Upload and Parsing

This module accepts the resume and prepares its text for analysis.

- **Input:** a resume file in PDF format sent to `POST /resume/analyze`.
- **Processing:** the file type and content are checked, the PDF is written to a temporary file, and the text, email address and phone number are extracted.
- **Output:** the extracted text and contact details, passed to the analysis module.

Uploads that are not PDF files, or that are empty, are rejected with an error message.

4.3 Module 3: Skill and Profile Analysis

The resume agent reads the extracted text and builds a profile of the student. It produces four kinds of result:

- The list of skills found in the resume.
- The strengths of the profile, based on the skills and experience found.
- The weaknesses and missing competencies, which show what to learn or add before applying.
- The recommended roles that suit the profile.

This profile is returned to the student and is also the input of the company-matching module.

4.4 Module 4: Company Matching

This module applies the algorithm described in Section 3.5 to the skills found in the resume.

- **Input:** the normalised resume skills and the company roles stored in the database.
- **Processing:** constraint propagation removes roles whose core skills are missing, and the remaining roles are scored by skill similarity and ranked.
- **Output:** a ranked list of company matches with recommendation insights, saved together with the analysis.

4.5 Module 5: Mock Interview Evaluation

This module scores a free-text answer. It supports five round types: aptitude, coding, technical, behavioural and communication. The request `POST /api/mock-interview/evaluate` carries the question, the answer and the round type. Figure 4.2 shows how the answer is processed.

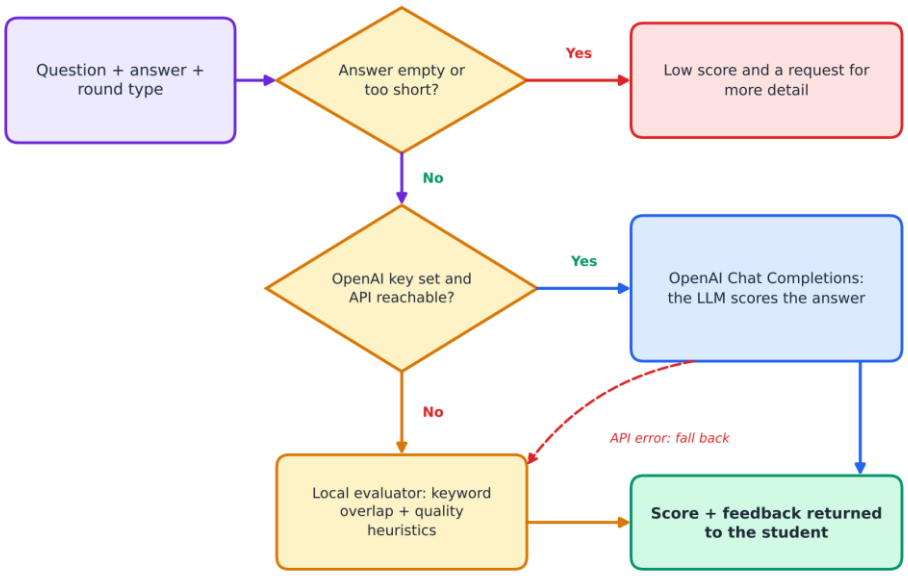


Fig. 4.2 Interview answer evaluation with local fallback

- **Input:** a question, the student's answer and the round type.
- **Processing:** empty or very short answers are filtered out; other answers are scored by the OpenAI API when a key is available, or by the local keyword-and-quality evaluator otherwise.
- **Output:** a score and written feedback.

4.6 Module 6: Coding Evaluation

For the coding round the student submits a solution to a coding problem. The request `POST /api/mock-interview/evaluate-code` evaluates the coding answer or its

test results and returns a score with feedback. As for text answers, the evaluation uses the OpenAI API when it is available and a local fallback when it is not.

4.7 Module 7: Tavus AI Interview

This module starts a conversational interview in which an AI interviewer talks to the student. The student enters a candidate name and an interview question. The request `POST /tavus/conversation` sends these values to the back end, which builds a custom interviewer persona and calls the Tavus API with the httpx client. The Tavus key is read from the TAVUS_API_KEY setting in the back-end environment file. Figure 4.3 shows the workflow.

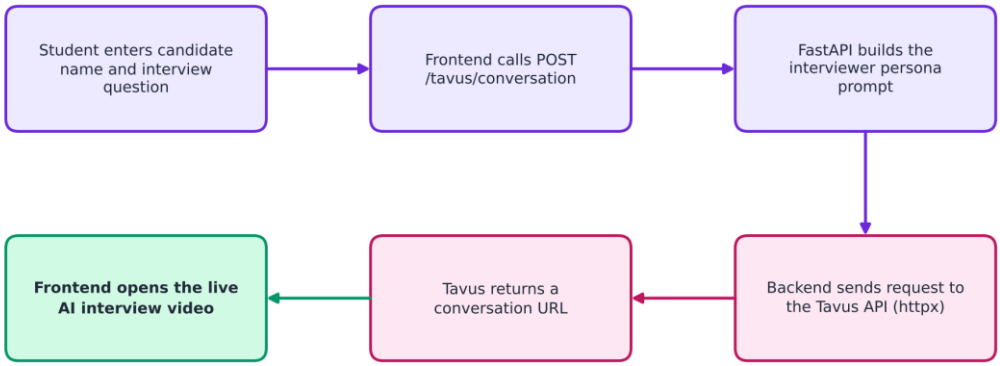


Fig. 4.3 Tavus AI interview workflow

4.8 Module 8: Dashboard and Student Experience

The front end gives the student one place to use all other modules. It is built with Next.js and has a responsive layout:

- **Home:** an overview with performance cards and progress states.
- **Resume Analysis:** upload of the PDF resume and display of the skills, strengths, weaknesses, roles and company matches.
- **Mock Interviews:** selection of an interview round, entry of answers and display of the score and feedback, together with the Tavus interview.

Some parts of the interface are still prototype components with placeholder content. They are listed in Chapter 6 as areas for improvement.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

The system was implemented and tested on a development machine. The back end runs with Uvicorn on port 8000 and the Next.js front end runs on its development server on port 3000. Table 5.1 lists the software environment.

Table 5.1 Software environment

Component	Version / tool
Python	3.10 or later
Back end framework	FastAPI with Uvicorn
Database	SQLite with SQLAlchemy
Node.js	20.9 or later (required by Next.js 16)
Front end	Next.js 16, React 19, TypeScript, Tailwind CSS, Framer Motion
Language model	OpenAI Chat Completions API (model set in OPENAI_MODEL)
Video interview	Tavus API (key set in TAVUS_API_KEY)

The database is filled by a seeding script. Table 5.2 shows the dataset that was used for the results in this chapter.

Table 5.2 Dataset used for the experiment

Item	Value
Companies	10 companies seeded by seed_companies.py
Company roles	4 roles with their required skills
Sample resumes	1 PDF resumes
Interview rounds	5 (aptitude, coding, technical, behavioural, communication)
Evaluation modes	OpenAI model Grok and local fallback
Demo users	3 registered users

Table 5.3 Configuration and run details

Parameter	Value
Back-end port	8000
Front-end port	3000
Resume format	PDF
Evaluator with an API key	OpenAI Chat Completions
Evaluator without an API key	Local keyword-and-quality evaluator

The experiment follows four steps: seed the database, start the back end and the front end, register and log in, and then upload a resume and attempt the interview rounds while inspecting the outputs.

5.2 Results

5.2.1 Student Dashboard

Figure 5.1 shows the dashboard. It has sections for Home, Resume Analysis and Mock Interviews, and uses performance cards and progress states to show the student's current position.

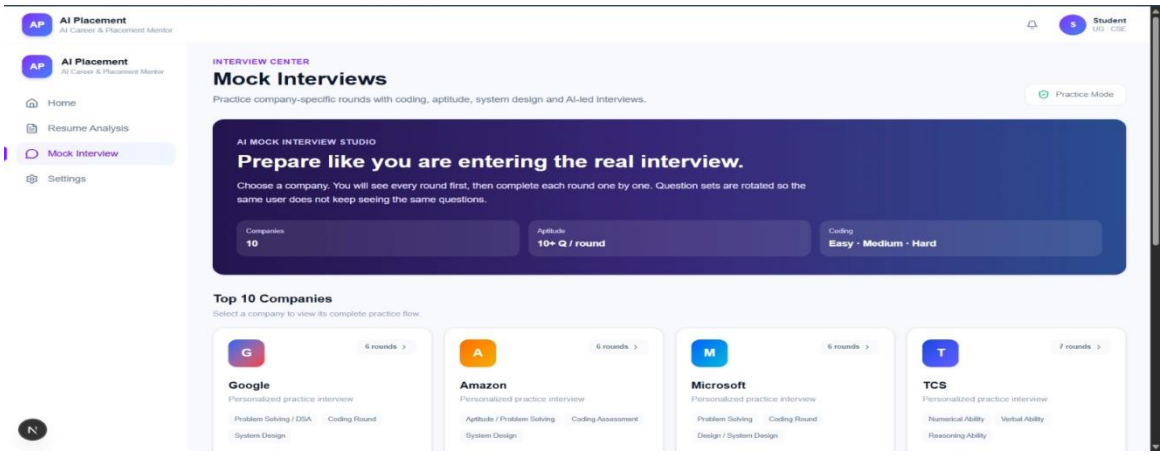


Fig. 5.1 Student dashboard (Home)

5.2.2 Aptitude Assessment Round

Figure 5.2 illustrates the aptitude assessment phase of the mock interview module. It features timed multiple-choice questions designed to test logical and quantitative reasoning, accompanied by a dynamic progress tracker showing completed and upcoming questions..

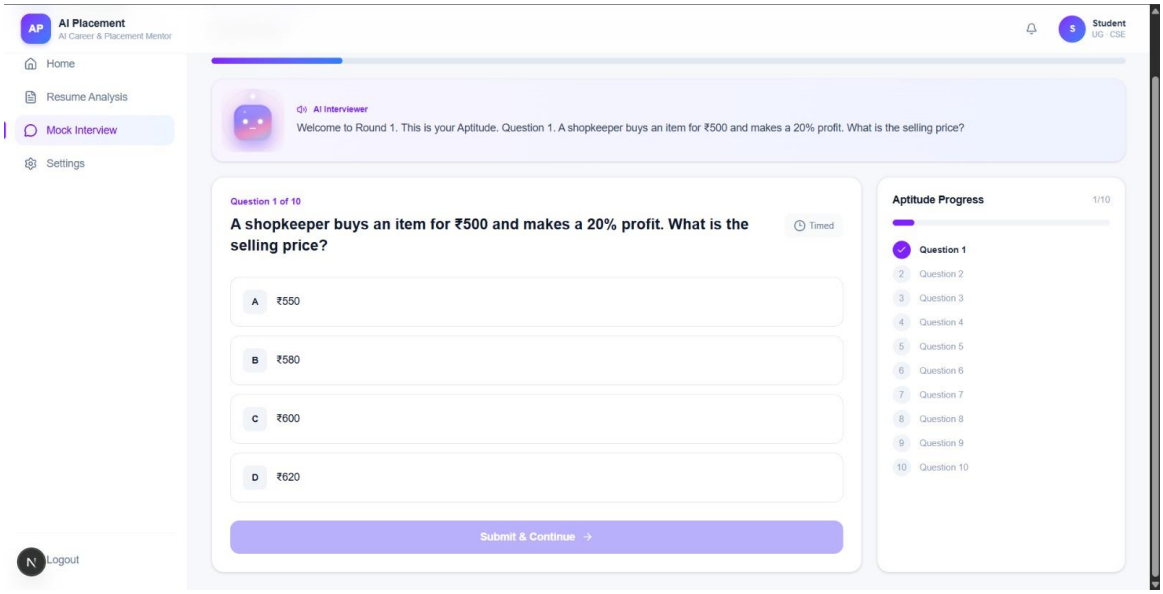


Fig. 5.2 Aptitude assessment interface

5.2.3 Company Matches

Figure 5.3 displays the interactive coding assessment module within the mock interview platform. It integrates a live code editor with syntax support, problem statement definitions, sample test cases, and automated execution testing to evaluate the candidate's programming proficiency.

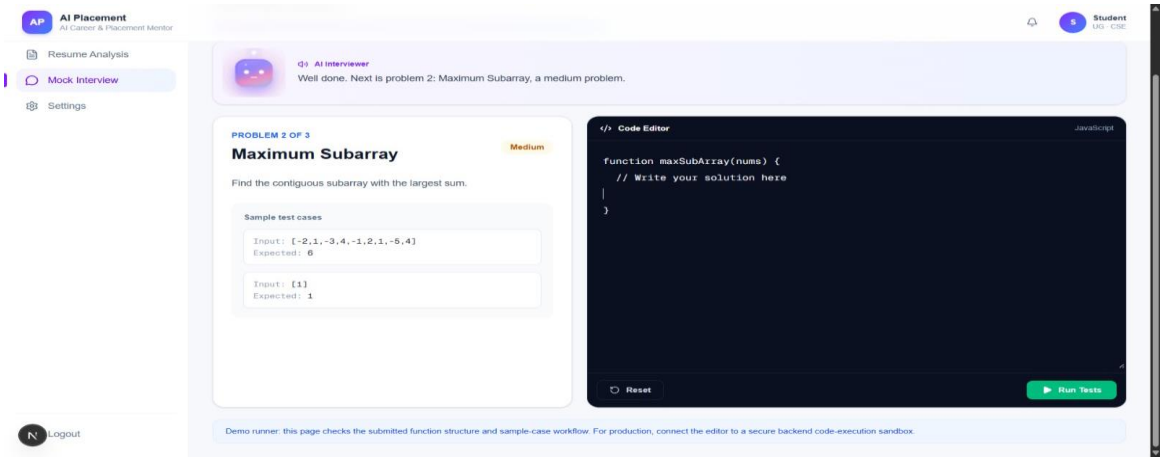


Fig. 5.3 Coding challenge and editor interface

5.2.4 Mock Interview Evaluation

Figure 5.4 shows a mock interview answer and the feedback returned by the evaluator. The student chooses a round type, writes an answer and receives a score with written feedback. The same screen is used when the local evaluator replaces the OpenAI API.

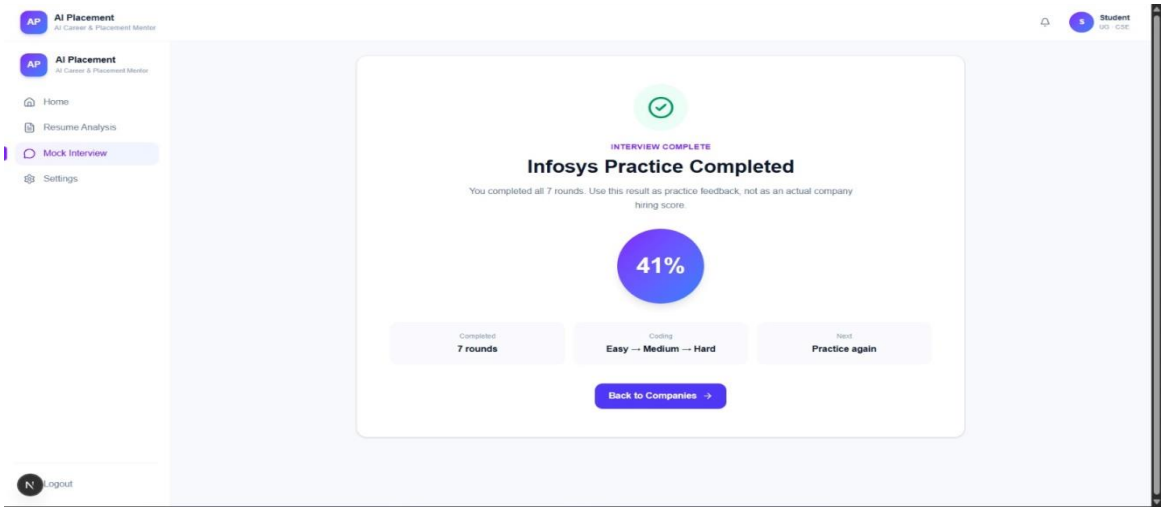


Fig. 5.4 Mock interview answer with AI feedback

5.2.5 Tavus AI Interview

Figure 5.5 shows a conversational interview with the Tavus AI interviewer. The student enters a name and an interview question, and the interviewer starts the conversation with the persona configured in the back end.

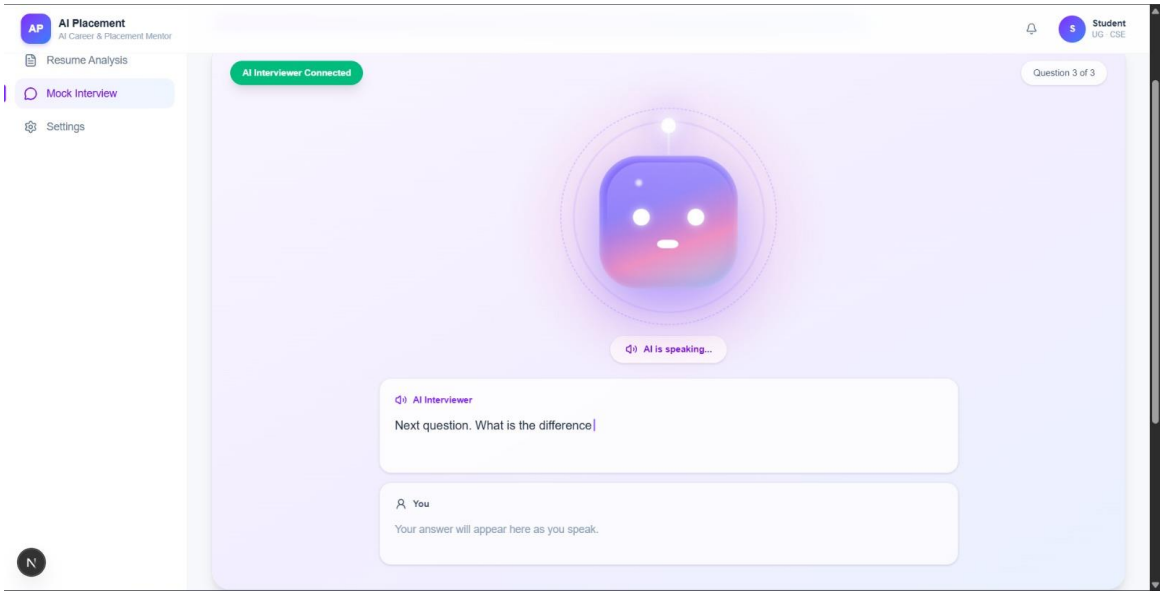


Fig. 5.5 Tavus AI interview session

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project built a complete placement preparation system around a practical workflow. A student can create an account, upload a PDF resume, receive an analysis of skills, strengths, weaknesses and recommended roles, see ranked company matches, and practise interview rounds with automated feedback and a conversational AI interviewer.

The company-matching step uses constraint propagation, a classical technique from the Foundations of Artificial Intelligence course, to remove company roles whose core skills are missing, and then ranks the remaining roles by skill similarity. The interview evaluator combines a language model with a local fallback, so the platform keeps working when the external service is not available.

The work shows that a small, well-understood classical method, combined with a language model and careful software design, can give students a useful and explainable assistant. The current version is a prototype, and the future work below describes the steps needed to take it towards production use.

6.2 Future Work

- Hash passwords with bcrypt or Argon2 and harden the token-based authentication before any production deployment.
- Add role-based access, for example for students, mentors and placement officers.
- Replace the fixed SQLite path with an environment-based database URL.
- Add a more robust resume parser or an OCR step for scanned resumes.
- Build a more advanced recommendation engine on a larger, real job dataset.
- Add analytics and progress tracking for each student across attempts.
- Use structured rubric scoring to improve the quality of interview feedback.
- Deploy the front end and the back end separately.
- Benchmark the platform on larger datasets and record analysis time and matching quality.

CHAPTER 7

REFERENCES

1. Bayer, M. (2012) ‘SQLAlchemy’, in Brown, A. and Wilson, G. (eds) The Architecture of Open Source Applications, Volume II, Lulu.com.
2. Brown, T.B., Mann, B., Ryder, N., et al. (2020) ‘Language Models are Few-Shot Learners’, Advances in Neural Information Processing Systems, Vol.33, pp.1877-1901.
3. Dechter, R. (2003) Constraint Processing, Morgan Kaufmann, San Francisco.
4. Fielding, R.T. (2000) Architectural Styles and the Design of Network-based Software Architectures, PhD Dissertation, University of California, Irvine.
5. Jones, K.S. (1972) ‘A Statistical Interpretation of Term Specificity and Its Application in Retrieval’, Journal of Documentation, Vol.28, No.1, pp.11-21.
6. Jones, M., Bradley, J. and Sakimura, N. (2015) ‘JSON Web Token (JWT)’, RFC 7519, Internet Engineering Task Force.
7. Mackworth, A.K. (1977) ‘Consistency in Networks of Relations’, Artificial Intelligence, Vol.8, No.1, pp.99-118.
8. OpenAI (2023) ‘GPT-4 Technical Report’, arXiv preprint arXiv:2303.08774.
9. Ricci, F., Rokach, L. and Shapira, B. (eds) (2015) Recommender Systems Handbook, 2nd edition, Springer, New York.
10. Russell, S. and Norvig, P. (2020) Artificial Intelligence: A Modern Approach, 4th edition, Pearson, Hoboken.
11. Salton, G. and Buckley, C. (1988) ‘Term-Weighting Approaches in Automatic Text Retrieval’, Information Processing and Management, Vol.24, No.5, pp.513-523.
12. Tsang, E. (1993) Foundations of Constraint Satisfaction, Academic Press, London.
13. Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023) ‘Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena’, Advances in Neural Information Processing Systems, Vol.36 (Datasets and Benchmarks Track).